Spiral Traverse

AlgoExpert · Medium · Arrays

The problem

- Read a (possibly non-square) matrix in clockwise spiral order.

Intuition

- Peel the matrix layer by layer: top row, right column, bottom row, left column, then shrink.

Brute force vs optimal

- Simulation with a visited matrix and direction turns → O(n) time, O(n) space
- Layer by layer with four boundaries → O(n), O(1) extra
- Both touch every cell once

Approach

- 1. Keep four boundaries
- 2. Walk the perimeter
- 3. Guard single rows or columns
- 4. Shrink and repeat

Worked example

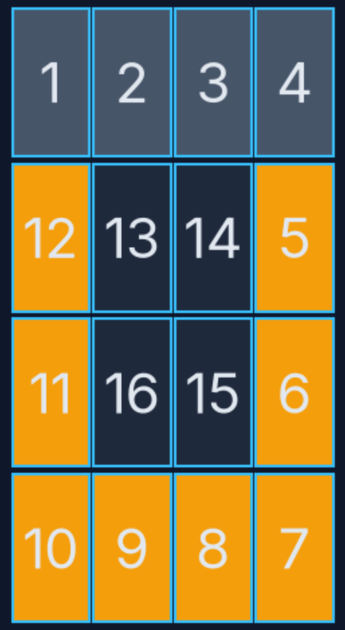
1	2	3	4
12	13	14	5
11	16	15	6
10	9	8	7

```
output:

1 2 3 4
```

Layer 0: top row

Worked example

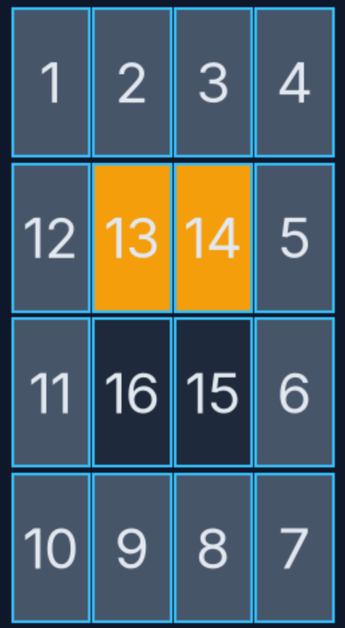



```
output:

1 2 3 4 5 6 7 8 9 10 11 12
```

Layer 0: right col, bottom row, left col

Worked example

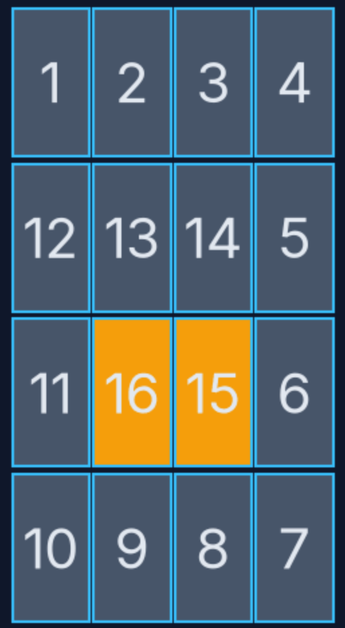


output:

1 2 3 4 5 6 7 8 9 10 11 12 13 14

Layer 1: top row

Worked example




```
output:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 1
```

Layer 1: right col, bottom row, left col

Java solution

```java
public class SpiralTraverse {
    public static List<Integer> spiralTraverse(int[][] m) {
        List<Integer> out = new ArrayList<>();
        if (m.length == 0) return out;
        int sr = 0, er = m.length - 1, sc = 0, ec = m[0].length - 1;
        while (sr <= er && sc <= ec) {
            for (int c = sc; c <= ec; c++) out.add(m[sr][c]);
            for (int r = sr + 1; r <= er; r++) out.add(m[r][ec]);
            for (int c = ec - 1; c >= sc && sr < er; c--) out.add(m[er][c]);
            for (int r = er - 1; r > sr && sc < ec; r--) out.add(m[r][sc]);
            sr++; er--; sc++; ec--;
        }
        return out;
    }
}
```

Complexity

- Time: O(n) cells
- Space: O(n) output

Edge cases & pitfalls

- Single row or single column — don't double count
- Non-square matrices
- Empty matrix

Interview tips & follow-ups

- Know recursive and iterative forms
- Follow-up: generate a spiral matrix 1…n^2

Recap

- Peel the matrix layer by layer: top row, right column, bottom row, left column, then shrink.
- Complexity: O(n) cells